Slip 10

Q1) A Compare AES with RSA based on type, key size, speed, and security, and explain key AES applications.

1. Difference Between AES and RSA

Basis	AES	RSA
1. Full form	Advanced Encryption Standard	Rivest–Shamir–Adleman
2. Type	Symmetric encryption	Asymmetric encryption
3. Keys used	Same secret key for encryption and decryption	Public key and private key
4. Key size	128, 192, or 256 bits	Commonly 2048 or 3072 bits
5. Speed	Very fast	Slower than AES
6. Security	Secure when used with a suitable mode and proper key management	Security depends on the difficulty of factoring large numbers and correct implementation
7. Key sharing	Secret key must be shared securely	Public key can be shared openly

8. Applications	File encryption, disk encryption, VPNs, and database protection	Key establishment, digital signatures, and older public-key encryption systems

2. Key Applications of AES

AES (Advanced Encryption Standard) is widely used to protect sensitive data because it provides strong security and fast encryption.

1. File and folder encryption: AES protects confidential documents and files from unauthorized access.

2. Disk encryption: It protects data stored on laptops, computers, and external storage devices.

3. Network security: AES is used in secure communication protocols and VPNs to protect transmitted data.

4. Wireless security: AES is used in modern Wi-Fi security standards, including WPA2 and WPA3.

5. Database security: AES encrypts sensitive information, such as personal records and financial data.

6. Cloud security: AES helps protect files and backups stored in cloud services.

7. Mobile applications: AES can protect locally stored sensitive information in applications.

3. Conclusion

AES is a fast symmetric encryption algorithm suitable for encrypting large amounts of data. RSA is an asymmetric algorithm that uses public and private keys for tasks such as key establishment and digital signatures. In practice, both may be used together: RSA can help establish or protect a secret key, while AES encrypts the actual data.

Q2) B Write a Python program to perform secure message communication using AES encryption and decryption

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

# Secret AES key (16 bytes = 128 bits)
key = b"1234567890123456"

# ---------------- SENDER ----------------

print("----- SENDER -----")

# Take message from sender
message = input("Enter message: ")

# Create AES cipher
cipher = AES.new(key, AES.MODE_ECB)

# Convert message into bytes and add padding
data = pad(message.encode(), AES.block_size)

# Encrypt message
encrypted_message = cipher.encrypt(data)

print("Encrypted Message:", encrypted_message.hex())


# ---------------- RECEIVER ----------------

print("\n----- RECEIVER -----")

# Create AES cipher for decryption
cipher = AES.new(key, AES.MODE_ECB)

# Decrypt message
decrypted_data = cipher.decrypt(encrypted_message)

# Remove padding
decrypted_message = unpad(decrypted_data, AES.block_size)
```

```
# Display original message
print("Decrypted Message:", decrypted_message.decode())
```

OR

Q2) B Write a Python program to generate MD5, SHA-1, and SHA-256 hashes of a file and verify its integrity.

```
import hashlib

# Accept file names
original_file = input("Enter original file name: ")
check_file = input("Enter file to verify: ")

# Read original file
f = open(original_file, "rb")
data1 = f.read()
f.close()

# Read file to verify
f = open(check_file, "rb")
data2 = f.read()
f.close()

# Calculate hash values of original file
md5_1 = hashlib.md5(data1).hexdigest()
sha1_1 = hashlib.sha1(data1).hexdigest()
sha256_1 = hashlib.sha256(data1).hexdigest()

# Calculate hash values of file to verify
md5_2 = hashlib.md5(data2).hexdigest()
sha1_2 = hashlib.sha1(data2).hexdigest()
sha256_2 = hashlib.sha256(data2).hexdigest()

# Display hash values
print("\nOriginal File:")
print("MD5    :", md5_1)
print("SHA-1  :", sha1_1)
print("SHA-256:", sha256_1)
```

```
print("\nFile to Verify:")
print("MD5    :", md5_2)
print("SHA-1  :", sha1_2)
print("SHA-256:", sha256_2)

# Verify file integrity
if md5_1 == md5_2 and sha1_1 == sha1_2 and sha256_1 == sha256_2:
    print("\nFile Integrity Verified")
    print("File is not modified.")
else:
    print("\nFile Integrity Failed")
    print("File is modified.")
```